Fundamentals of System Security
(COMSM0122)

Workshop 2: Breaking and Understanding Byzantine Fault Tolerance

Prerequisite:
(1) A code editor such as Visual Studio Code
(2) Node.js (version 18 or newer)
(3) Basic familiarity with JavaScript
(4) A GitHub account
(5) Completed the lecture on Byzantine Fault Tolerance

Workshop Learning Objectives
This workshop will provide you with practical experience of Byzantine Fault Tolerant (BFT) systems by allowing you to observe, modify, and attack a simplified JavaScript implementation of a consensus protocol. Through a series of guided exercises, you will explore how consensus is achieved in the presence of faulty or malicious participants, investigate the role of voting, quorums, cryptographic signatures, and commit certificates, and develop an understanding of why BFT protocols trust protocol rules and evidence rather than individual nodes.

The workshop also develops the security-focused mental models introduced in the lecture. You will experiment with Byzantine behaviour such as equivocation, key compromise, Sybil attacks, quorum manipulation, and denial-of-service scenarios to understand the assumptions upon which BFT guarantees depend. By the end of the workshop, you should be able to reason about the distinction between protocol correctness and system security, identify common attack surfaces in BFT-based systems, and critically evaluate how violations of trust assumptions can undermine consensus guarantees even when the protocol itself is correctly implemented.

You are encouraged to work in teams of 2-3 students.

Setup
In this workshop, you will work with a simplified JavaScript-based Byzantine Fault Tolerant (BFT) simulator designed specifically for learning and experimentation. The simulator models a network of distributed nodes that exchange messages, vote on proposed values, form quorums, and commit decisions. Unlike a production BFT implementation, the simulator has been intentionally simplified to make the core concepts of trust, voting, cryptographic verification, and consensus easier to understand and explore.

The simulator contains several predefined scenarios and attack modules that allow you to observe how the protocol behaves under both normal and adversarial conditions. You will progressively enable Byzantine behaviour, introduce equivocation, simulate key compromise, create Sybil identities, and manipulate quorum rules to explore the assumptions that underpin BFT systems. Throughout the workshop, you are encouraged to modify the code, inspect message flows, and analyse protocol

outcomes to understand not only how consensus is achieved, but also how and why it can fail when its trust assumptions are violated.

Directory Structure

```
bft-lab/
├── package.json
├── simulator.js
├── node.js
├── network.js
├── crypto.js
├── attacks.js
└── scenarios/
    ├── honest.js
    ├── byzantine.js
    ├── equivocation.js
    ├── key_theft.js
    ├── sybil.js
    └── quorum_tampering.js
```

Component Responsibilities

(1) Node (`node.js`)
Represents an individual participant in the distributed system. Each node:
- Maintains a local state
- Generates votes
- Signs messages
- Verifies messages from other nodes
- May behave honestly or maliciously

Example:

```
Node A
  ├── ID
  ├── State
  ├── Secret Key
  ├── vote()
  └── verify()
```

(2) Network (`network.js`)
Simulates communication between nodes.
Responsibilities:
- Broadcast messages
- Deliver votes
- Record message traffic
- Simulate delays or dropped messages

Attack exercises can modify this layer to model: Denial-of-Service attacks, Network partitions, and Message delays.

(3) Cryptography Layer (`crypto.js`)
Provides message authentication.

Responsibilities:
- Sign messages
- Verify signatures
- Detect forged messages

Example message:
```
{
  "node":"A",
  "value":"100",
  "signature":"abc123..."
}
```

This component supports the mental model: *Every message requires proof*

(4) Consensus Engine (`simulator.js`)
Implements the protocol's core decision logic.
Responsibilities:
- Collect votes
- Count agreements
- Determine quorum
- Commit decisions
- Produce commit certificates

Example

```
if (votes >= quorumSize()) {
    commit(value);
}
```
This layer embodies the rule: **Trust the protocol, not individual participants.**

(6) Quorum Engine
The quorum engine is part of the simulator's consensus logic and determines how many votes are required before agreement can be reached.

For a 4-node system: `quorum = 3`

You will later tamper with this logic to observe how incorrect quorum calculations can violate BFT safety guarantees.

This component reinforces: **Quorums are overlapping truth sets.**

(7) Audit Log and Evidence Store
Records:
- Votes
- Signatures
- Commit certificates

- Consensus decisions

  Example:
  ```
  COMMIT CERTIFICATE
  Value = 100
  Signatures:
  A
  B
  C
  ```

The audit log demonstrates that BFT systems are not merely replicating state, but rather `State + Evidence.`
This reflects the workshop mental model: **You are building a distributed audit trail.**

(8) Attack Framework (`attacks.js`)
Provides reusable mechanisms for introducing Byzantine behaviour. Attack modules include:

Attack	Purpose
Byzantine Vote	Node deliberately votes for an incorrect value
Equivocation	Node sends different values to different peers
Key Theft	Attacker impersonates a legitimate validator
Sybil Attack	Attacker creates multiple fake validators
Quorum Tampering	Protocol safety is weakened by changing quorum rules

The attack framework allows students to explore how BFT assumptions can be violated without modifying the entire simulator.

(9) Scenario Files
Each workshop exercise is provided as a self-contained scenario.
- `honest.js` (Honest consensus execution.)
- `byzantine.js` (Single faulty node.)
- `equivocation.js` (Conflicting signed messages.)
- `key_theft.js` (Validator impersonation.)
- `sybil.js` (Membership manipulation.)
- `quorum_tampering.js` (Safety violation through incorrect quorum mathematics.)

Design Philosophy
This simulator is intentionally educational rather than production-grade. The architecture has been designed to make protocol assumptions visible, encourage experimentation, and allow students to directly observe the consequences of Byzantine behaviour. Throughout the workshop, you should view the simulator not simply as a consensus implementation, but as a laboratory for exploring trust, cryptographic evidence, quorum formation, and the security assumptions that underpin Byzantine Fault Tolerant systems.

Installation
Download the workshop repository from Blackboard and install the required dependencies by running the following shell command:

```
npm install
```

Running a Scenario
Each task is provided as a separate scenario file. To execute a scenario, run:

```
node scenario/honest.js
```

You can replace `honest.js` with any of the other scenario files:
- `node scenarios/byzantine.js`
- `node scenarios/equivocation.js`
- `node scenarios/key_theft.js`
- `node scenarios/sybil.js`
- `node scenarios/quorum_tampering.js`

Workshop Approach
The workshop follows an experimentation-based approach. Rather than implementing a BFT protocol from scratch, your task is to observe protocol behaviour, introduce faults, and reason about the resulting outcomes. For each task, consider the following questions:

- What assumptions does the protocol make?
- Which participants are trustworthy?
- What evidence is required before accepting a decision?
- What happens when a node behaves maliciously?
- Does the protocol preserve safety?
- Does the protocol preserve availability?
- Which mental model from the lecture best explains the observed behaviour?

As you progress through the tasks, you should begin to recognise that BFT is fundamentally about establishing truth despite deception, using protocol rules, cryptographic evidence, and quorum overlap rather than trusting individual participants.

Part 1: The Trust Problem
The purpose of this part is to introduce the most important mindset in Byzantine Fault Tolerance: **Do not trust individual nodes. Trust the protocol**.
You begin by observing a network in which all participants behave honestly. The goal is to understand the normal flow of a consensus decision, how votes are collected, and how a committed value is chosen. You should become familiar with the simulator output, vote tallying process, and quorum formation before exploring malicious behaviour.

Tasks

1. Run the `honest.js` scenario.
2. Observe how each node votes for the proposed value.
3. Examine the vote tally and quorum calculation.
4. Record the committed value.
5. In maximum of 200 words, explain why consensus was reached.

Reflection Questions

- Why was the proposed value accepted?
- Did any individual node determine the outcome?
- What evidence exists that agreement occurred?
- What role did the quorum play?

You should understand that consensus emerges from protocol rules and agreement among participants rather than trust in any single node.

Part 2: Byzantine Behaviour

The purpose of this part is to demonstrate that BFT systems can continue operating correctly even when a participant behaves maliciously or provides incorrect information.

You will introduce a Byzantine node that deliberately votes for an incorrect value while the remaining nodes continue to behave honestly. The exercise illustrates the practical meaning of the `3f + 1` rule and shows how honest participants can outvote faulty ones.

Tasks

1. Run the `byzantine.js` scenario.
2. Observe how the malicious node votes differently from the rest of the system.
3. Compare the vote tally with the previous exercise.
4. Identify the committed value.
5. In maximum of 200 words, explain the Byzantine behaviour and how the system was able to behave correctly.

Reflection Questions

- Why did consensus still succeed?
- Did the protocol need to determine exactly which node was malicious?
- How many Byzantine nodes can this deployment tolerate?
- What assumption is the protocol making?

You should recognise that BFT protocols do not need to identify malicious nodes. Instead, they rely on quorum mathematics and trust assumptions to maintain correctness.

Part 3: Equivocation

This part focuses on one of the most important forms of Byzantine behaviour: *equivocation*.

Unlike simply lying, an equivocating node sends different information to different participants. This is often more dangerous because each message may appear individually valid. Students will observe how a single node can create multiple conflicting realities within the system.

Tasks

1. Run the `equivocation.js` scenario.
2. Observe the different values sent to different recipients.
3. Inspect the messages generated by the malicious node.
4. Compare the information received by each participant.
5. In maximum of 200 words document your observation

Reflection Questions

- Are the messages correctly signed?
- Why is equivocation more dangerous than a simple incorrect vote?
- Could an individual recipient detect the attack?
- Why must nodes exchange evidence with one another?

You should understand that cryptographic authenticity alone is insufficient. A message can be authentic yet still be malicious.

Part 4: Cryptographic Evidence and Key Theft

This part explores the role of cryptography in BFT systems and demonstrates the consequences of key compromise.

You will observe how signatures provide proof of origin and integrity. You will then perform a simulated key theft attack and see how an attacker can impersonate a legitimate participant once a private key is compromised.

Tasks

1. Review the vote-signing process.
2. Run the `key_theft.js` scenario.
3. Observe how forged votes are generated using a stolen private key.
4. Compare legitimate and forged messages.
5. In maximum of 200 words document your observation

Reflection Questions

- Can honest nodes distinguish a forged message from a legitimate one?
- Why does signature verification succeed?
- What assumptions does the protocol make about cryptographic keys?
- What happens if those assumptions become invalid?

You should understand that BFT protocols depend heavily on secure key management. Consensus cannot protect against compromised credentials.

Part 5: Sybil Attacks and Membership Management

The purpose of this part is to demonstrate that consensus relies not only on protocol correctness but also on proper identity and membership management.

You will simulate a Sybil attack by introducing multiple attacker-controlled validators into the system. This exercise highlights the distinction between the consensus problem and the admission-control problem.

Tasks

1. Run the `sybil.js` scenario.
2. Observe how new nodes are added to the validator set.
3. Identify which nodes are controlled by the attacker.
4. Consider the effect on voting power and quorum calculation.
5. In maximum of 200 words document your observation

Reflection Questions

- Who decides which nodes may participate?
- Why does consensus not automatically solve the identity problem?
- How do blockchain systems attempt to prevent Sybil attacks?
- What happens if an attacker controls a majority of identities?

You should understand that many real-world BFT deployments depend on external governance and admission-control mechanisms.

Part 6: Quorum Reasoning and Safety

The purpose of this part is to explore why quorum calculations are fundamental to BFT safety.

You will modify the quorum rules used by the simulator and observe how seemingly small changes can completely undermine the protocol's safety guarantees. This exercise reinforces the importance of quorum overlap and the `3f + 1` requirement.

Tasks

1. Run the `quorum_tampering.js` scenario.
2. Examine the modified quorum definition.
3. Observe how two conflicting values can satisfy the weakened quorum rule.
4. Compare the outcome with the original simulator behaviour.
5. In maximum of 200 words document your observation

Reflection Questions

- Why does the modified quorum rule fail?
- What property has been lost?
- Can multiple values reach quorum simultaneously?
- Why must future quorums overlap with previous quorums?

You should understand that BFT safety depends on carefully chosen quorum mathematics. Incorrect quorum rules can lead to contradictory decisions and violation of consensus guarantees.

Part 7: Breaking BFT

This part challenges you to apply everything learned throughout the workshop.

You will investigate the simulator, combine attacks, modify protocol assumptions, and attempt to violate consensus guarantees. The goal is not merely to execute attacks but to identify which trust assumptions have failed and explain why the protocol behaves as observed.

Tasks

1. Select one or more attack scenarios.
2. Modify protocol parameters or node behaviour.
3. Attempt to:
    - create conflicting commits,
    - prevent consensus,
    - impersonate validators,
    - manipulate membership,
    - weaken quorum guarantees.
4. Document the attack and its impact.

Reflection Questions

- Which trust assumption was violated?
- Is this a protocol, infrastructure, governance, or application issue?
- Did safety fail?
- Did availability fail?
- Could the attack occur in a real deployment?

You should be able to critically evaluate the assumptions behind BFT systems, distinguish between protocol correctness and overall security, and explain how attacks against infrastructure, governance, cryptography, or implementation can undermine a mathematically correct consensus protocol.

Part 8: Hardening a BFT System

The purpose of this part is to move from attack analysis to system defence. In the previous parts, you explored how various attacks can undermine the assumptions of a Byzantine Fault Tolerant system. In this final exercise, you will implement and evaluate mitigations that strengthen the protocol against those attacks.

You should recognise that BFT protocols are not inherently secure; rather, they remain secure only when their underlying assumptions are protected. The goal is therefore to identify weak points and introduce additional safeguards that improve resilience against malicious participants, compromised credentials, faulty configurations, and governance failures.

Scenario

You have inherited a BFT implementation that has been subjected to:

- Byzantine behaviour
- Equivocation
- Key compromise
- Sybil attacks
- Quorum manipulation

Your task is to modify the simulator and implement one or more defensive mechanisms. A successful solution should either:

- prevent the attack,
- detect the attack,
- reduce its impact,
- or make recovery easier.

Task 8.1: Detect Equivocation

Prevent a node from successfully presenting different values to different peers without being detected.

Current Weakness

A malicious node can send:

```
A → 100
B → 50
C → 75
```

and each message is properly signed.

Hardening Strategy

Add an evidence log.

```
equivocationLog = {
```

```
  nodeId,
  value,
  recipient
}
```
Compare messages received from the same sender.
If:
```
Node D
Value 100

and

Node D
Value 50
```
appear in the same round: `EQUIVOCATION DETECTED`

You will learn that signatures provide authenticity, not consistency.

Task 8.2: Strengthen Key Protection
Your goal is to reduce the impact of key theft.

Current Weakness
An attacker stealing `secret-A` can impersonate `Node A` indefinitely.

Hardening Strategy
Introduce key rotation. Example: `node.rotateKey();` every N rounds.

Alternative: Implement key revocation. `revoke("A");` and reject future messages.

The key lesson here is that cryptography is only as strong as key management.

Task 8.3: Prevent Sybil Attacks
The goal of this task is to control membership.

Current Weakness
Attackers can create `X1, X2, X3, X4` and gain voting power.

Hardening Strategy
Introduce validator registration. Example:
```
approvedValidators = [
  "A",
  "B",
  "C",
  "D"
];
```

Reject unknown participants.
```
if ( !approvedValidators.includes(vote.nodeId)){
  rejectVote();
}
```

The key lesson here is to understand the importance of admission control and governance.

Task 8.4: Protect Quorum Rules
Your goal is to prevent accidental or malicious weakening of quorum mathematics.

Current Weakness
A developer modifies:

```
size() {
return 2;
}
```

causing safety violations.

Hardening Strategy
Add sanity checking. Example:

```
validateQuorum() {

   const minimum = Math.floor((2 * totalNodes) / 3) + 1;

   if (quorumSize < minimum) {
      throw Error("Unsafe quorum size");
   }
}
```

The key lesson here is to understand that protocol parameters are security-critical.

Task 8.5: Build a Commit Certificate
The goal of this task is to strengthen auditability.

Current Weakness
Consensus only stores:
COMMIT 100

Hardening Strategy
Store evidence.

```
certificate = {
  value: 100,
  signatures: [
    sigA,
    sigB,
    sigC
  ]
};
```

Output:

```
COMMIT CERTIFICATE
```

```
Value = 100
Signed by:
A
B
C
```

In doing this task, you will reinforce the mental model: **Consensus is a distributed audit trail.**

Part 9 (Optional Extension): Secure the Simulator

This optional extension activity allows you to move beyond identifying attacks and begin thinking like security engineers. Rather than simply observing vulnerabilities, you will design and implement improvements to strengthen the Byzantine Fault Tolerant system against one or more attack scenarios explored during the workshop.

There is no single correct solution. You are encouraged to evaluate the trust assumptions of the simulator, identify weaknesses, and implement appropriate mitigations. Solutions may focus on protocol changes, cryptographic protections, validation mechanisms, governance controls, or monitoring and detection capabilities.

Suggested Extensions

You may choose one or more of the following:

- Detect and report equivocation attempts.
- Implement validator admission control.
- Add key revocation or key rotation.
- Introduce commit certificates and audit logs.
- Add quorum validation checks.
- Create a mechanism for blacklisting malicious nodes.
- Implement basic message rate limiting.
- Design and implement their own defence.

Deliverables

For each enhancement:

1. Describe the attack being addressed.
2. Explain the security assumption being strengthened.
3. Describe the implementation changes.
4. Demonstrate the mitigation in action.
5. Discuss any remaining limitations.

Reflection Questions

- Which attacks can be prevented completely?
- Which attacks can only be detected?
- How does the mitigation affect protocol complexity?
- Does the mitigation introduce any new assumptions?
- What additional defences would a production BFT system require?

At the end of this task, you should be able to critically evaluate the security assumptions of a BFT system and propose practical improvements that increase resilience against attacks while preserving protocol correctness.

Submission
Submit your implementation of both tasks as a zip file on Blackboard.
Notes:

1. Submission points will only be open on the day of the workshop.
2. You must be physically present in the lab to submit (except permitted otherwise)
3. You will not gain marks for submitting.
4. Submissions help us differentiate learning and engagement levels when marking both group and individual coursework.

   For all submissions before group assessment, **you will lose 1 mark for each missed submission up to a maximum of 5 marks as an indication of lack of engagement in learning**.
   For all submissions after group assessment and before individual assessment, **you will lose 1 mark for each missed submission up to a maximum of 5 marks as an indication of lack of engagement in learning**.

5. Finally, push your `bft-lab` project folder to GitHub. Ensure that the project is associated with appropriate documentation and a README.md description. Tag your project appropriately, including the workshop date and name. Finally, invite the GitHub user systemsecuritybcsg to your pushed project. This GitHub handle is managed by the unit director.